Project Defense Script

Local Job Connector — spoken script for presenting/defending this project (demo day, supervisor review, panel). ~8–10 minutes, plus a Q&A section with prepared answers for the hard questions.

---

1. Opening (30s)

"My project is **Local Job Connector**, a mobile-first marketplace that connects people in Ghanaian cities — Accra, Kumasi, Tema — who need casual work done, like cleaning, delivery, repairs or event help, with people looking for that work. It's built in Flutter so it runs on web, Android and iOS from one codebase, backed by Firebase for authentication, data and hosting."

2. The problem (30–45s)

"Right now this kind of work is arranged informally — word of mouth, WhatsApp groups, physical noticeboards. There's no trusted, local way for a seeker to find nearby jobs, or for someone posting work to reach the right people and vet applicants. The goal isn't to replace this entirely, but to give both sides clear listings, a direct application flow, and in-app chat so the whole transaction doesn't require handing out a phone number to a stranger up front."

3. Who it's for (20s)

- **Seeker** — browses or maps nearby jobs, applies, tracks application status, chats with the poster.
- **Poster** — posts a job, reviews applicants, accepts or rejects, marks the job filled or complete.
- **Admin** — moderates content, manages users, handles reports.

"A single account can switch between seeker and poster roles."

4. Live demo (3–4 min)

Walk through one end-to-end path, narrating as you go:

1. Sign up / sign in with real Firebase Auth — show a wrong password being rejected (this matters, see Q&A below).
2. Post a job as a poster (title, category, pay, city, location).
3. Switch account/role, browse the feed, apply as a seeker.

4. Switch back, review the applicant, accept them.
5. Open the chat tied to that job, send a message both ways.
6. Show the report flow: report a job, then open the **admin dashboard** and show it landing in the Reports queue, and resolve it there.

"Everything you just saw is live against the real Firebase project, not a mock — Firestore, Firebase Auth, and a static admin dashboard that reads the same database."

5. Architecture, briefly (1 min)

"The client is Flutter using the `provider` package for state. Screens talk to Providers, Providers talk to a repository interface, and the repository has two implementations — a real Firestore-backed one, and an in-memory one used only in tests. That separation is what let me write and run tests without hitting live Firebase for every check."

"Data lives in Firestore: `users`, `jobs`, `applications`, `chats` with a messages subcollection, and now `reports` for moderation. Access is controlled by Firestore security rules — for example, only the two participants in a chat can read or write it; only the job's poster can change an application's status."

6. What's done vs. in progress (1 min)

Be direct about this — it's a strength, not a weakness, if framed as engineering discipline:

"This project is being built in phases, and I track that explicitly in a roadmap doc alongside a risk register. Phase 1 — making the backend real instead of relying on fake fallback data — is done and verified live with an automated end-to-end suite. Phase 2, core product completion, is partly done: password reset, account deletion, and the reports flow are live and rule-tested. Still open for Phase 2: phone verification, pagination, push notifications, and reporting individual messages. Phase 3 is a stronger admin system with custom-claim roles and an audit log, and Phase 4 is launch hardening — accessibility, privacy policy, Ghana's Data Protection Act registration, load testing."

7. Closing (20s)

"The core loop — post a job, apply, accept, chat, report — works end to end on live infrastructure today. The remaining work is documented, prioritized, and scoped, not open-ended. I'm happy to go deeper into the security model, the data model, or the roadmap."

Q&A — prepared answers for likely hard questions

"Is this secure? I heard early versions had fake logins."

"Yes, that was real and it's in my own risk register, not something I'm hiding. Early on, sign-in fell back to a fake user if Firebase wasn't initialized, and the backend was never actually wired up in `main.dart`, so the app ran on in-memory data silently. Both are fixed and verified: auth now comes from Firebase's real `authStateChanges` stream, a wrong password is rejected, and there's no fake-user fallback in the release build. I track it as risks R1 and R2 in my security doc with a status of 'fixed, tests pass.'"

"What stops one user from reading another user's private chats or data?"

"Firestore security rules, enforced server-side — not something the client can bypass. Chats are scoped to an array of participant UIDs; only those two UIDs or an admin can read or write. Jobs are publicly readable but only the owning poster can update them, and `posterId` is immutable after creation. I have rule tests running against the Firebase emulator that assert a user *cannot* read another user's chat or edit someone else's job."

"How do you store passwords / handle auth?"

"I don't store passwords — Firebase Auth handles credential storage and hashing entirely; the app only ever sees a signed-in user's UID and ID token. The web admin dashboard is currently gated by a Firestore document lookup, which I've flagged as an interim measure — the plan is to move it to Firebase custom claims, which are enforceable inside security rules and can't be spoofed by writing to a document."

"What about user privacy — phone numbers, Ghana's data law?"

"Good catch, and it's a known gap, not an oversight: early on a job document publicly exposed the poster's phone number. The fix in the data model is to keep the phone number on the user's private profile and only reveal it after an application is accepted. Ghana's Data Protection Act, 2012 also requires registering as a data controller, a published privacy policy, consent capture at sign-up, and a self-service delete-my-account flow — the deletion flow is already built and live; the rest is scoped into the pre-launch hardening phase."

"Why Flutter and Firebase instead of a custom backend?"

"Managed services mean no servers to operate, patch or scale myself, and Firebase's free tier plus pay-as-you-grow pricing fits a pre-revenue project. Flutter gives one codebase for web, Android and iOS, which matters for a two-sided marketplace where I need both seekers and posters reachable on whatever device they have. The tradeoff is less control over backend logic, which is why trust-sensitive logic — role changes, status transitions — lives in Firestore rules and planned Cloud Functions rather than in the client."

"How do you know the app actually works, not just looks like it works in a demo?"

"I have an end-to-end test suite — five scripts, thirty-five checks — that runs against the live deployed site, covering sign-up, sign-in rejection, posting a job, applying, accepting, and chatting both directions. That suite is what caught several real bugs: hard-coded fake user IDs that made writes get silently attributed to the wrong account, and a provider that only read from local cache so posters never saw their applicants. Those are fixed and re-verified live."

"What's the single biggest risk left?"

"Firestore rules that properly scope `applications` and `chats` to their participants are written and unit-tested against the emulator, but hadn't been deployed to production as of my last review — so that's the top open item, along with rotating credentials that were exposed during development. Both are first on my Phase 0/1 checklist before I'd call this launch-ready."

"What would you do with another month?"

"Deploy and verify the tightened Firestore rules in production, finish loading/error/empty states on the remaining screens, add push notifications, and start the Phase 4 hardening work — accessibility pass, privacy policy, and a proper security review against my risk register before even considering opening it to real users."

Notes for delivery

- Keep the demo path short and rehearsed — one job, one applicant, one chat, one report. Don't improvise new paths live.
- If something breaks live, don't panic-debug on stage: say what *should* happen, note it, and move to Q&A — the test suite and docs back you up.
- Lead with what's real and working before what's planned; it's more credible than a feature list.

- Don't claim things that are "Open" in the risk register as done. If asked, admit the gap and name the planned fix — that's a stronger answer than hiding it.